Lecture 1: Introduction to Software Architecture

Module 1 | Software Architecture

IIIT Nagpur

Software Architecture Describe how the system is divided into components and how those components interact with each other.

What is Software Architecture?

- Software Architecture refers to the high-level structure of a software system. It describes how a system is organized into major components and how these components interact with each other.
- Unlike coding, which focuses on writing algorithms, software architecture focuses on system-wide decisions that influence the entire software lifecycle.
- For example, deciding whether an application should follow a layered architecture or a client-server architecture is an architectural decision.

Definition by Shaw and Garlan

- According to Mary Shaw and David Garlan, software architecture is concerned with the selection of architectural elements, their interactions, and the constraints on those elements.
- These constraints help provide a framework within which system requirements can be satisfied and future design decisions can be made.
- This definition highlights that architecture is more about structure and relationships than about implementation details.

Definition by Bass, Clements, and Kazman

- Bass, Clements, and Kazman define software architecture as the structures of a software system, which comprise software elements, their externally visible properties, and the relationships among them.
- Externally visible properties include performance characteristics, communication protocols, and interfaces.
- This definition emphasizes that architecture is what stakeholders can observe and reason about early in development.

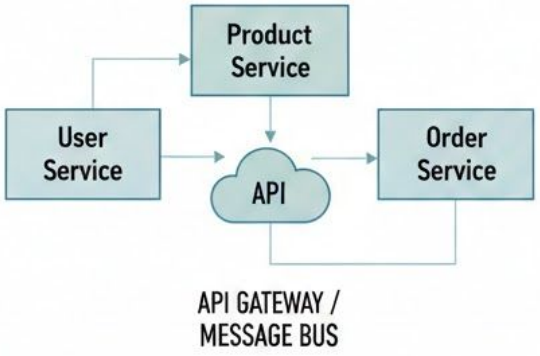
Architecture vs Design

- Software architecture and software design are closely related but distinct concepts.
- Architecture deals with high-level system organization, while design focuses on the internal details of individual components.
- For example, deciding to use a microservices architecture is an architectural decision, while designing classes inside a service is a design activity.

SOFTWARE ARCHITECTURE vs. SOFTWARE DESIGN

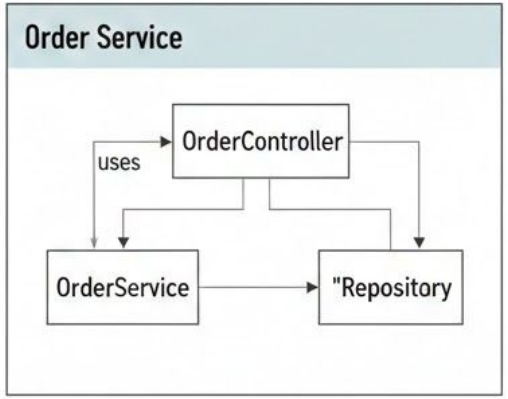
VS.

SOFTWARE ARCHITECTURE
(HIGH-LEVEL SYSTEM ORGANIZATION)



DECISION: USE MICROSERVICES ARCHITECTURE

SOFTWARE DESIGN
(INTERNAL COMPONENT DETAILS)



DESIGN: CLASSES WITHIN "ORDER SERVICE"

Refinment Process →

Civil Architecture Analogy

- Software architecture can be compared to civil architecture used in building construction.
- A building blueprint defines the structure before construction begins, just as software architecture defines the structure before coding starts.
- Multiple views such as floor plans and elevations in civil architecture correspond to different architectural views in software systems.

Elements of Software Architecture

- The core elements of software architecture are components, connectors, interfaces, and configurations.
- These elements together describe both the static structure and dynamic behavior of a system.
- Understanding these elements is essential for analyzing and designing large-scale systems.

Components

- A component is a unit of computation or a data store that performs a specific function within the system.
- Components have clearly defined interfaces through which they interact with other components.
- Examples of components include databases, authentication modules, and payment processing services.

Component vs Object

- Objects are usually small-grained entities created and destroyed frequently during program execution.
- Components are larger-grained and have a longer lifecycle, often existing for the entire duration of the system.
- A component may internally contain many objects working together.

Connectors

- Connectors are architectural elements that enable communication and coordination between components.
- They define how components interact rather than what computation is performed.
- Examples include procedure calls, message queues, and event buses.

Types of Connectors

Connectors may be implicit or explicit

1- Implicit:

An implicit connector is a connector that is not represented as a separate architectural element.

It is hidden inside the programming language or execution mechanism.

Key Characteristics of Implicit Connectors

- Not shown as separate boxes or elements
- Communication is embedded inside components
- Easy to use but harder to analyze
- Common in traditional programming

Types of Implicit Connectors

1- Procedure / Function Calls

One component calls a function or method of another component.

2- Shared Data / Shared Variables

Multiple components communicate by reading and writing shared data.

2- Explicit Connectors

An explicit connector is a connector that is modeled as a separate, first-class architectural element.

Key Characteristics of Explicit Connectors

- Shown explicitly in architecture diagrams
- Treated as independent elements
- Easier to analyze, modify, and replace
- Very common in distributed systems

Types of Explicit Connectors

1-Message Passing

Components communicate by sending messages.

2-Event-Based Connectors

One component generates an event, and other components react to it.

3-Pipes and Filters

Output of one component becomes input of another through a pipe.

4-Client–Server Middleware

A middleware layer manages communication between client and server.

5-Shared Repositories

Components communicate through a central data store that is explicitly modeled.

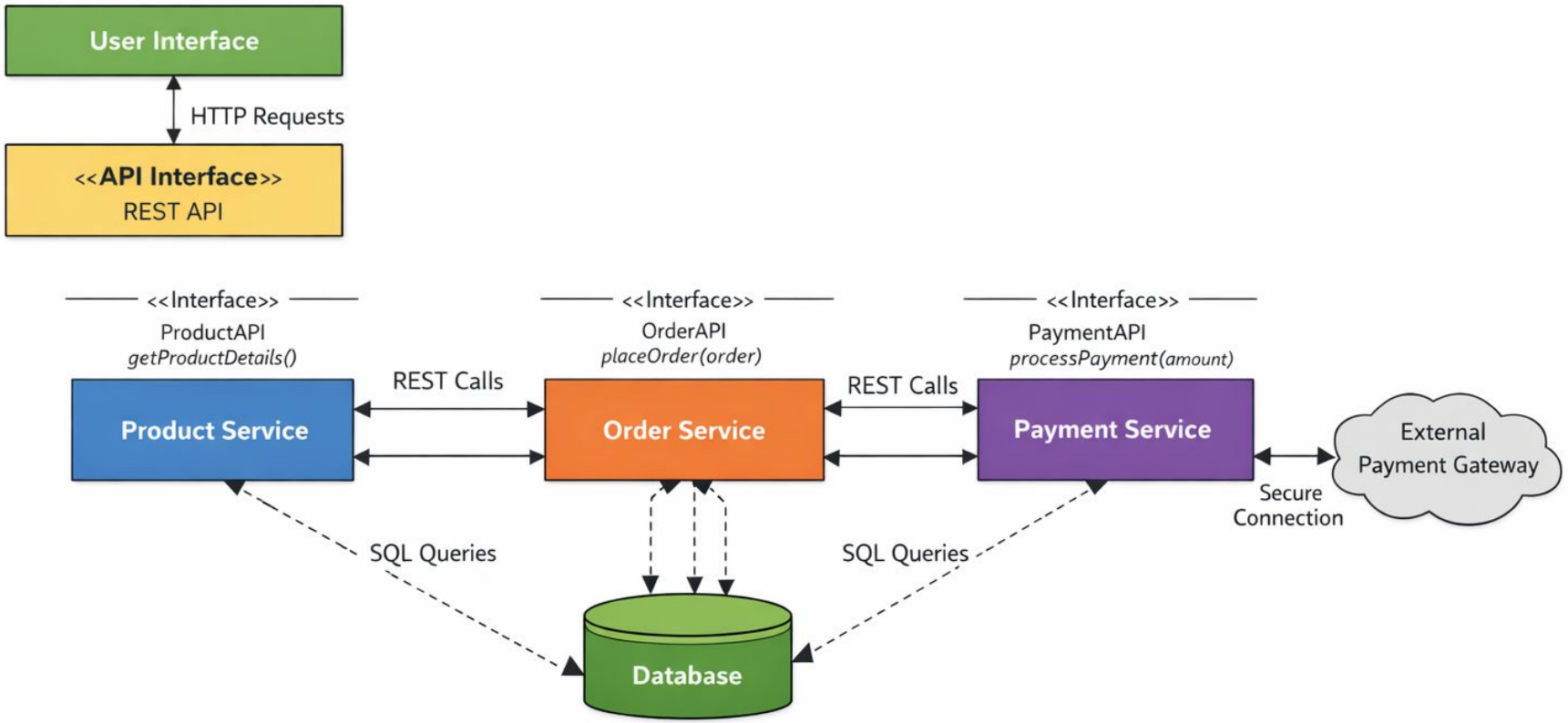
Interfaces

- Interfaces define the services provided and required by components.
- They act as contracts that specify how components can interact without revealing internal details.
- Well-defined interfaces improve reusability and interoperability.

Configurations and Topology

- A configuration represents the overall arrangement of components and connectors in a system.
- Topology refers to the structural layout, such as linear, layered, or network-based arrangements.
- Configurations help in analyzing compatibility, performance, and scalability.

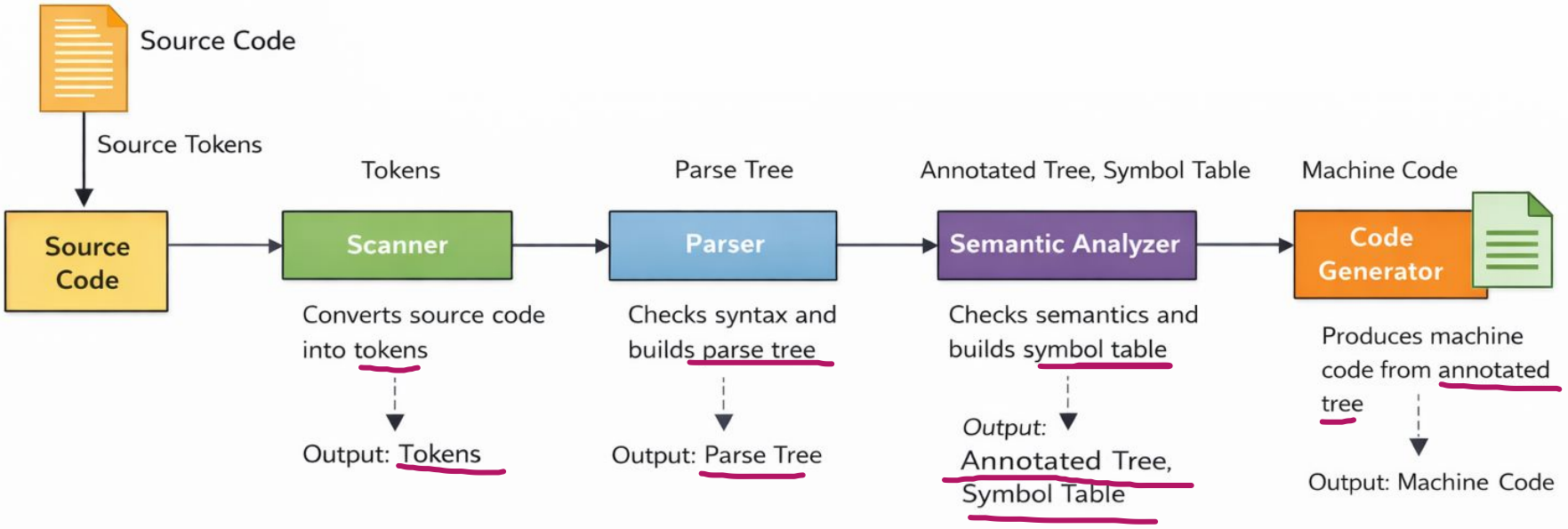
Software Architecture Example



Example: Compiler Architecture

- A compiler is a classic example used to explain software architecture concepts.
- It consists of components such as scanner, parser, semantic analyzer, and code generator.
- Each component performs a specific task and passes its output to the next component.

Compiler Architecture Example



Why Software Architecture Matters

- Software architecture has a strong impact on system qualities such as performance, scalability, and maintainability.
- Architectural decisions are difficult and costly to change once the system is implemented.
- A well-designed architecture reduces long-term maintenance effort and development cost.